Deployment

Option 1 — Vercel + Neon (recommended)

1. **Database:** create a Neon project (region *ap-south-1 Mumbai* if available). Copy the **pooled** connection string → `DATABASE_URL`, the **direct** one → `DIRECT_URL` (used by `prisma migrate`).
2. **Project:** import the repo in Vercel (framework: Next.js). Set env vars from `.env.example`: `APP_URL`, `AUTH_SECRET`, `BRAND_NAME`, `ADMIN_EMAIL`, `DATABASE_URL`, `DIRECT_URL`, SMTP_*, `EMAIL_FROM`, `SMS_PROVIDER`, MSG91_*, `CRON_SECRET`, optional `GOOGLE_MAPS_API_KEY`, RAZORPAY_*. **Do not** set `EXPOSE_DEV_OTP`.
3. **Migrate + seed once** from your machine (or CI):

```
DATABASE_URL="<direct url>" npx prisma migrate deploy
DATABASE_URL="<direct url>" ADMIN_EMAIL=you@company.com npm run db:seed   # no SEED_DEMO in prod
```

4. **Cron:** on Vercel Pro copy `vercel.cron.example.json` to `vercel.json` (per-minute). On Hobby, call `GET /api/cron/tick` every minute from an external scheduler with `Authorization: Bearer $CRON_SECRET`. Dispatch also self-advances when customers poll tracking, so a delayed cron degrades gracefully.
5. **Domain & email:** add your domain; configure SPF/DKIM for the SMTP provider (Amazon SES Mumbai, Resend, SendGrid…).

Option 2 — Docker on a VPS / any container host

```
docker compose up -d --build        # db, mailpit, migrate+seed, app, cron
```

For production, edit `docker-compose.yml`: remove `mailpit` and point `SMTP_*` at your provider, set strong `AUTH_SECRET` / `CRON_SECRET`, use a managed Postgres (or keep `db` with a backed-up volume), and put Caddy/Nginx/Traefik in front for TLS. The image is a non-root Next.js standalone server with a `/api/health` healthcheck. Migrations run via the `migrate` one-shot service (`prisma migrate deploy`).

SMS (MSG91)

Register on DLT, get the OTP template approved, then set `SMS_PROVIDER=msg91`, `MSG91_AUTH_KEY`, `MSG91_SENDER_ID`, `MSG91_OTP_TEMPLATE_ID`. Non-OTP SMS (booking updates) use a DLT *flow* template with one variable → `MSG91_FLOW_TEMPLATE_ID` (`VAR1` = message). Until configured, SMS are logged (mock) and visible in **Admin → Notification log**.

Ops checklist

- Health: `GET /api/health` (200 + DB check). Logs: stdout. Add Sentry/Datadog via their Next.js SDKs if desired.
- Backups: Neon PITR or `pg_dump` for self-hosted.

- Secrets rotation: changing `AUTH_SECRET` invalidates sessions and pending OTPs.
- Public OSM endpoints (tiles, Nominatim, OSRM) have fair-use limits — for real traffic host your own, or switch to Google/Mappls.

Firebase Phone Auth (optional)

1. Firebase console → create/choose a project → **Authentication → Sign-in method → Phone → Enable**.
2. **Authentication → Settings → Authorized domains**: add your production domain (e.g. `towingservice.vercel.app`).
3. **Project settings → Your apps → Web app** and copy the config into these environment variables on Vercel/Render, then redeploy: `FIREBASE_PROJECT_ID`, `FIREBASE_API_KEY`, `FIREBASE_AUTH_DOMAIN`, `FIREBASE_APP_ID`.
4. Behaviour: phone numbers are verified with Firebase (invisible reCAPTCHA). If Firebase cannot deliver the SMS (quota, billing, network) and an email address is known, the code is emailed instead. Email OTP is always available. India SMS on Firebase needs the Blaze plan; the free Spark plan only allows a small number of SMS.